

PROJETO FINAL

Trabalho Prático – Modelos, Métodos e Técnicas da Engenharia de Software – Lucas Goulart Silva

Pedro Chrtistovam Marques
RA: 124117694

Gabriel Alves Bragança
RA: 124221803

Vinicius Francisco Silva
RA: 1252211622482

Pedro Henrique Silva LopesS
RA: 11914300

João Vitor Romani Nogueira de Paula
RA: 124110378

DEFINIÇÃO DO PROBLEMA

O problema para o qual foi proposta uma solução está relacionado ao ambiente de trabalho do aluno Pedro Christovam, junto à MGI — empresa vinculada ao Governo do Estado de Minas Gerais, responsável pela gestão e administração dos imóveis registrados em território mineiro.

CONTEXTO

A empresa realiza vistorias em imóveis localizados em todo o território do Estado de Minas Gerais, com o objetivo de avaliar propriedades que posteriormente poderão ser destinadas a processos de leilão público, incluindo imóveis residenciais, comerciais e terrenos.

Para a execução dessas atividades, os colaboradores responsáveis pelas vistorias realizam visitas presenciais aos locais e elaboram uma ficha técnica contendo informações relevantes sobre o imóvel. Esses dados são utilizados para apresentação e análise durante os processos de leilão.

(Para fins acadêmicos e de segurança da informação, optou-se por utilizar apenas informações consideradas básicas e não sigilosas neste levantamento, evitando qualquer associação com dados internos ou confidenciais da empresa.)

PROBLEMA REAL

Atualmente, os colaboradores responsáveis pelas vistorias realizam todo o processo de coleta de informações de forma manual, utilizando papel e caneta para o preenchimento das fichas técnicas dos imóveis. Esse método apresenta diversos problemas operacionais, como falhas de escrita, informações incompletas, letra ilegível, perda de documentos físicos e danos causados pelo transporte ou armazenamento inadequado das fichas. Essas situações comprometem diretamente a continuidade do processo de avaliação e leilão dos imóveis. Em muitos casos, os problemas identificados tornam necessária a realização de uma nova vistoria no local, gerando retrabalho para a equipe, aumento de custos operacionais e desconforto para o proprietário do imóvel. Além disso, nem sempre é possível obter uma segunda oportunidade para realizar a vistoria novamente, o que pode ocasionar atrasos ou inviabilizar o andamento do processo.

SOLUÇÃO ELABORADA

Com base nos requisitos apresentados no enunciado, foi proposta a criação de um sistema que poderá funcionar tanto em plataforma web quanto em aplicativo mobile, com o objetivo de auxiliar os colaboradores responsáveis pelas vistorias de imóveis.

O sistema permitirá que os usuários realizem o preenchimento digital das informações necessárias diretamente durante a vistoria, além da possibilidade de anexar fotografias do local vistoriado. Após o preenchimento, os dados serão enviados imediatamente por meio do próprio sistema, garantindo maior agilidade e segurança no armazenamento das informações.

Além disso, o sistema será responsável pela geração automática de um documento em formato PDF, permitindo o registro e a conferência do material enviado. Essa funcionalidade proporcionará maior controle e rastreabilidade dos dados coletados.

Com a implementação da solução, espera-se reduzir significativamente falhas humanas relacionadas ao preenchimento manual, evitar perdas ou danos de documentos físicos e minimizar a necessidade de uma segunda visita ao imóvel, tornando o processo mais eficiente, seguro e organizado.

2. LEVANTAMENTO E ANÁLISE DE REQUISITOS

Com base nisso, foi identificada a necessidade de um sistema web/mobile que permita o preenchimento digital das informações da vistoria, anexação de fotografias e envio imediato dos dados para o sistema.

Os principais atores envolvidos são os colaboradores vistoriadores, responsáveis pelo preenchimento das informações, e os administradores do sistema, responsáveis pelo gerenciamento e acompanhamento das vistorias.

As funcionalidades previstas incluem:

- Login de usuários;
- Cadastro de vistorias;
- Preenchimento digital da ficha técnica;
- Upload de fotos;
- Geração automática de PDF;
- Consulta e armazenamento das vistorias realizadas.

A solução busca reduzir falhas humanas, evitar perda de documentos, diminuir retrabalho e melhorar a organização e eficiência do processo de vistoria.

DIAGRAMA DE CLASSES (SIGEVI)

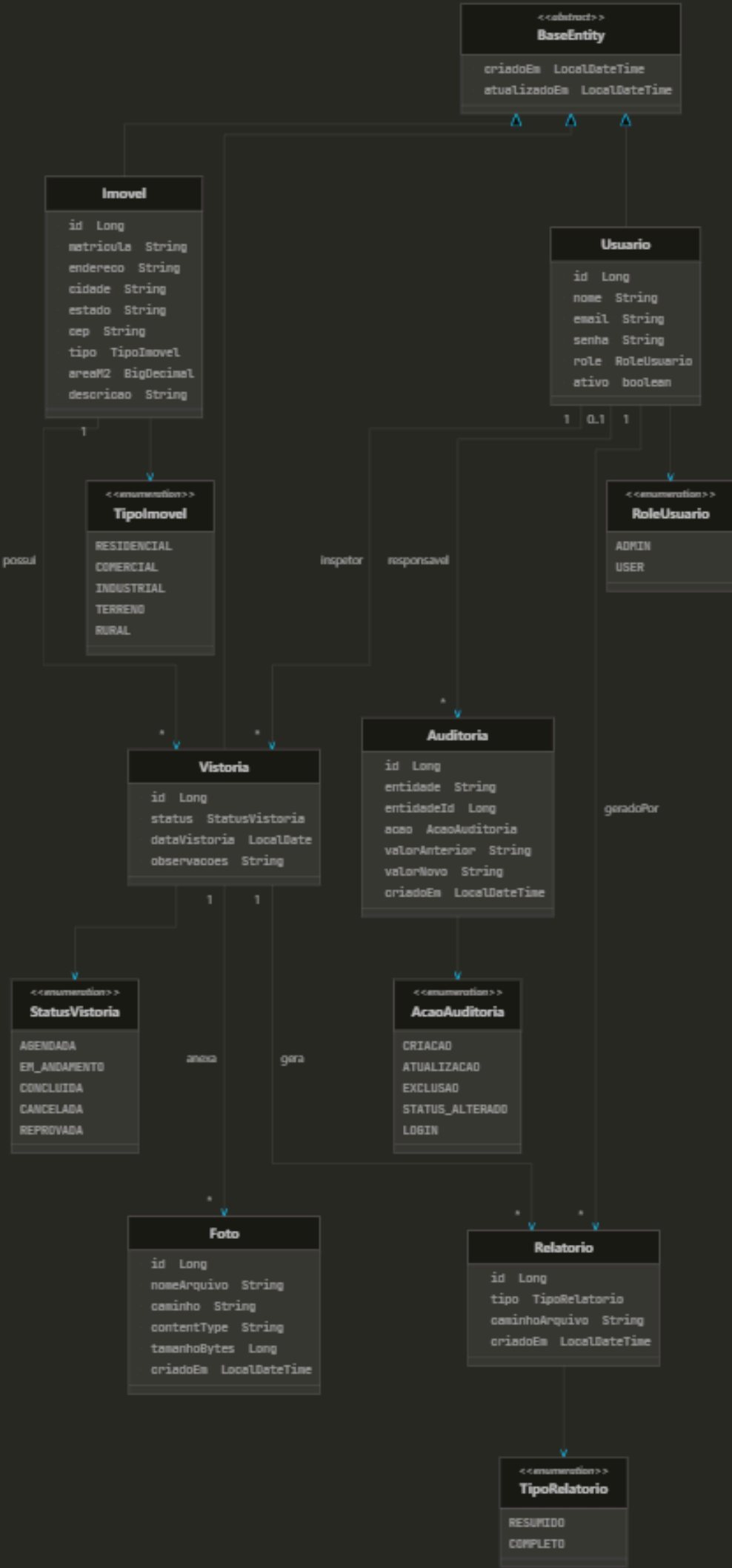
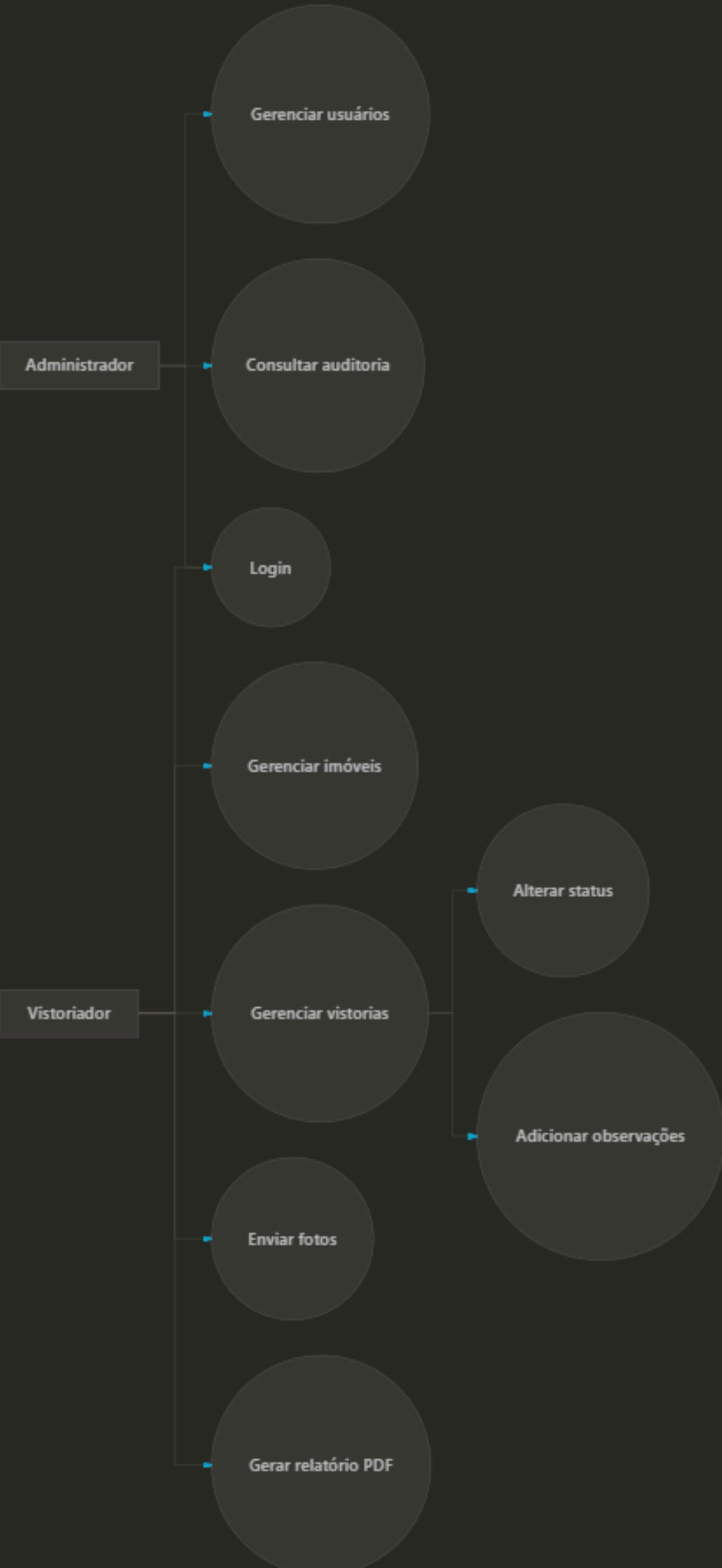
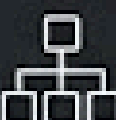


DIAGRAMA DE CASOS DE USO (SIGEVI)



O QUE SIGNIFICA CADA PRINCÍPIO SOLID E ONDE ESTÁ NO SIGEVI

Princípio (SOLID)	O que significa (bem direto)	Onde está no SIGEVI (exemplos)	Arquivo(s) Exemplo(s)	Descrição do Arquivo
 SRP (Single Responsibility)	Cada classe tem um único motivo principal pra mudar.	Classes focadas em uma única função.	validator/ImagemValidator.java, service/*Service.java	Valida upload; Regras de negócio.
 OCP (Open/Closed)	Dá pra estender sem ficar alterando classe existente.	Novo tipo de relatório = nova Strategy (sem mudar o Context).	pattern/strategy/RelatorioStrategyContext.java, pattern/strategy/RelatorioCompletoStrategy.java	Contexto que usa estratégias; Exemplo de implementação.
 LSP (Liskov Substitution)	Implementações podem ser trocadas sem quebrar o uso.	RelatorioResumido e Completo funcionam como RelatorioGeracaoStrategy.	RelatorioGeracaoStrategy (Interface), AuditoriaListener (Interface)	Contratos para substituição; Contrato de evento de auditoria.
 ISP (Interface Segregation)	Interfaces pequenas, sem "método sobrando".	pattern/observer/AuditoriaListener.java (1 método), repositórios Spring Data.	pattern/observer/AuditoriaListener.java (1 método), repositórios/UsuarioRepository.java	Exemplo de interface pequena (onAuditoriaEvent); Repositório focado.
 DIP (Dependency Inversion)	Camadas altas dependem de abstrações, não detalhes.	service/*Service injeta *Repository (interfaces); security utiliza interfaces.	service/*Service.java (Interface usage), config/SecurityConfig.java	Classe que usa interfaces injetadas; Configuração de segurança usando abstrações.

NO SIGEVI, QUAIS SÃO OS TESTES UNITÁRIOS QUE JÁ EXISTEM;

Eles estão em: sigevi/src/test/java/br/com/sigevi/

.validator/StatusVistoriaValidatorTest: testa se as transições de status são aceitas/rejeitadas corretamente.

.validator/ImagemValidatorTest: testa validação de formato/tamanho/arquivo vazio no upload.

service/UsuarioServiceTest: testa regra de e-mail duplicado e cadastro.

service/ImovelServiceTest: testa matrícula duplicada e cadastro.

service/VistoriaServiceTest: testa alteração de status (e caso “não encontrado”).

service/AuthServiceTest: testa login com credenciais válidas e inválidas.

security/JwtTokenProviderTest: testa geração e validação do JWT.

FLUXO DO SISTEMA NA PRÁTICA



OBRIG

ADO